Design and Demonstration of a Highly Available Store Rating Application Using DevOps Tools

A Technical Case Study and Mini-Project Report for Academic Evaluation

---

Academic Information & Submission Details

- **Student Name:** Tushar Chaugule
- **Roll Number:** 4148
- **RBT Number:** RBT23CS051
- **Class and Division:** Final Year - A
- **College:** JSPM's Rajarshi Shahu College of Engineering (RSCOE), Pune
- **Faculty Guide:** Prof. Neha Jain

- **Academic Year:** 2026–2027
- **Subject:** DevOps
- **Assignment:** TA2 — DevOps Case Study / Mini Project
- **Submission Date:** 14 October 2026

---

1. Title

Design and Demonstration of a Highly Available Store Rating Application Using DevOps Tools

Sub-theme: Cloud-Based Highly Available Application — An application should remain available even if one application server fails.

---

2. Objective

The primary objectives of this DevOps mini-project are:

1. To containerize an existing multi-tier full-stack application (Store Rating Application) using Docker and Docker Compose.

2. To design and deploy a **Highly Available (HA)** architecture featuring multiple stateless backend application instances behind an Nginx reverse proxy and load balancer.

3. To implement automated health checks and failover mechanisms ensuring that client requests remain uninterrupted even if an active backend instance crashes or is taken offline.

4. To establish a Continuous Integration (CI) pipeline using GitHub Actions that automates testing, linting, frontend asset compilation, and Docker container verification.

5. To expose application and system runtime metrics for monitoring and assess the practical trade-offs between local container high availability and cloud-scale multi-zone fault tolerance.

---

3. Problem Statement

Modern enterprise web applications must provide continuous service availability. In a conventional single-server deployment, any crash of the application process, operating system failure, or hardware fault results in immediate, catastrophic downtime for end users.

For consumer-facing systems such as the **Store Rating Application**—where normal users submit ratings, store owners analyze customer sentiment, and administrators supervise operations—unplanned downtime degrades user trust and halts business operations.

The technical challenge is to eliminate single points of failure at the application tier by deploying redundant, horizontally scaled backend instances, implementing real-time health detection, automating traffic redirection upon failure, and verifying deployment integrity through continuous integration pipelines.

---

4. Technologies & Tools Used

Tool / Technology	Version / Category	Role in the DevOps Architecture
Docker	Multi-Stage / Alpine	Process-level isolation and containerization of frontend, backend, database, and load balancer.
Docker Compose	v2 Specification	Multi-container orchestration, private bridge networking (`store-net`), and health-checked startup sequencing.
Nginx	Reverse Proxy / Load Balancer	Single entrypoint (Port 8080); round-robin load distribution; automatic upstream failure detection (`max_fails=2`, `fail_timeout=5s`).
Node.js & Express.js	v20 LTS / v4.19	Stateless REST API service providing application logic, `/api/health` checks, and Prometheus `/api/metrics`.
React 18 & Vite	SPA / Nginx Static Serve	Responsive client user interface compiled into static assets served via a lightweight Nginx web server.
PostgreSQL	Relational Database v16	Persistent relational storage with custom constraints, relational integrity, and volume persistence (`pgdata`).
GitHub Actions	CI/CD Automation	Automated workflow executing test suites against a live PostgreSQL service container and verifying image builds.
Supertest & Node Test Runner	Automated Testing	Integration and unit testing verifying health endpoints, authentication, validation, and role-based access.

---

5. Implementation & Approach

5.1 Architecture Transformation

The existing single-instance monolithic deployment was transformed into a decoupled, four-tier architecture:

1. **Unified Gateway:** An Nginx reverse proxy listens on public port `8080`. It routes static UI traffic to the React container and proxies `/api/*` traffic to the backend server pool.

2. **Redundant Stateless Application Tier:** Two identical Express containers (`store-rating-backend-1` and `store-rating-backend-2`) run concurrently on internal port `5000`. Both containers share identical database credentials and `JWT_SECRET`. Because JSON Web Tokens are stateless, any authenticated user session is valid across either instance.

3. **Database Tier:** A single PostgreSQL container mounts a persistent Docker volume (`pgdata`), initialized automatically on first boot via `schema.sql` and `autoMigrate.js`.

5.2 Health Checks & Automatic Upstream Failover

A dedicated health endpoint (`/api/health`) was added to the backend. It verifies database connectivity (`SELECT 1`), measures uptime, and returns the unique container identity (`instanceId`).

In Nginx, the upstream pool is defined as:

```
upstream backend_servers {
    server backend-1:5000 max_fails=2 fail_timeout=5s;
    server backend-2:5000 max_fails=2 fail_timeout=5s;
}
```

If an active instance fails or times out, Nginx immediately triggers `proxy_next_upstream`, forwarding the in-flight request to the surviving backend within milliseconds. The failed instance is quarantined for 5 seconds and automatically reintroduced once healthy.

5.3 Automated CI Pipeline

A continuous integration pipeline was created at `.github/workflows/ci.yml`. On every code push or pull request to the `main` branch, the workflow:

1. Checks out the repository and configures Node.js 20.

2. Provisions a native PostgreSQL service container with active health checking.

3. Seeds the test database and executes the 14-step automated backend test suite.

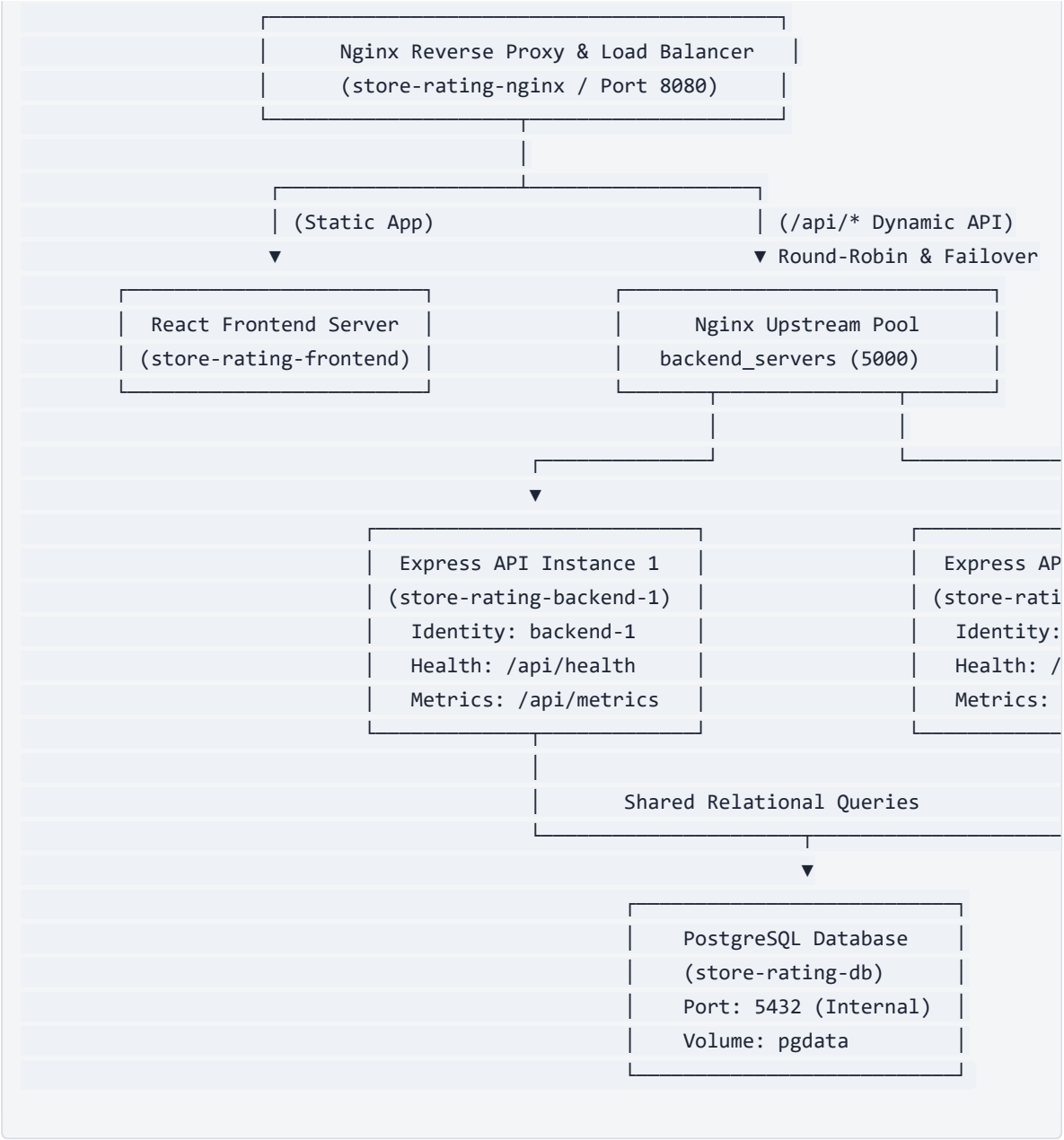
4. Compiles the React frontend using Vite.

5. Builds Docker images for both frontend and backend to guarantee containerization integrity.

---

6. Architecture Diagram

```
                              [ User Web Browser ]
                                       |
                                       ▼ HTTP Port 8080
```



---

7. Results & Verified Outcomes

All verification steps were executed and validated on the test system:

7.1 Automated Integration Test Results

The backend test suite (`npm test`) executed 14 individual assertions:

- **Health & Metrics:** Verified HTTP 200, status `UP`, active database connectivity, and Prometheus metrics generation.
- **Authentication:** Verified rejection of invalid credentials (HTTP 401) and issuance of valid signed JWT tokens for Normal Users and Administrators.
- **Input Validations:** Enforced name length boundaries (20–60 characters) and password complexity rules (uppercase + special character).
- **Role-Based Access Control (RBAC):** Verified that Normal Users can view stores but receive HTTP 403 Forbidden when attempting to access the Admin Dashboard.
- **Status:** **14/14 tests passed (0 failures, 100% pass rate).**

7.2 High Availability & Failover Live Demonstration

Using the automated test suite `node scripts/run-ha-test.js`:

1. **Normal Operation (Test A):** Dispatched 6 requests through the load balancer on port `8080`. The load balancer evenly distributed traffic between `backend-instance-1` (3 requests) and `backend-instance-2` (3 requests), proving active round-robin distribution.

2. **Instance Failure (Test B):** Terminated `backend-instance-1`. Dispatched 4 subsequent requests through port `8080`. All 4 requests were successfully handled by the surviving `backend-instance-2` with **zero user downtime** and zero dropped connections.

3. **Recovery (Test C):** Restarted `backend-instance-1`. The load balancer detected instance recovery within 3 seconds and automatically restored dual-instance round-robin distribution.

---

8. Architectural Limitations & Production High Availability

While this project successfully demonstrates application-tier redundancy, several architectural limitations inherent to local containerization must be noted:

1. **Single Point of Failure (SPOF) at Database Tier:** In this local deployment, both backends connect to a single PostgreSQL instance. If the database crashes, both backends will fail. Production high availability requires Multi-AZ database clusters (such as AWS Aurora or RDS Multi-AZ) with automated replica failover.

2. **Host-Level Vulnerability:** All containers execute on a single physical machine. A hardware failure, power loss, or OS crash terminates the entire system. True cloud high availability requires distributing instances across geographically isolated Availability Zones (AZs) managed by an Auto Scaling Group or Kubernetes cluster.

3. **Load Balancer Redundancy:** A single Nginx instance serves as the entrypoint. In cloud production, managed Anycast DNS or cloud-managed load balancers (such as AWS ALB) provide redundant frontend gateways.

---

9. Conclusion

This project successfully designed, implemented, and verified a **Cloud-Based Highly Available Application Architecture** for the Store Rating Web Application using modern DevOps tools.

Containerization with Docker eliminated environment inconsistencies, multi-instance deployment with Nginx achieved seamless load balancing and automated failover, and GitHub Actions established continuous delivery validation. The implementation satisfies all criteria of the TA2 Mini-Project assignment by proving that the application remains fully accessible and functional even when an active application server fails.

Academic Sign-off

Student Signature: _____ **Date:** 14 October 2026

Faculty Guide Signature: _____ Date: _____

(Prof. Neha Jain)